Instruction and Data Memory Hierarchy: A Simple Harvard and Shared-Memory Comparison

Akshat Chauhan, Ajeesh Amreet, Vaibhav,
Hardik Maheshwari and Mitul Bhatia
Team HEXARCH, Section A — MCA Project 9, 2026
Newton School of Technology, Rishihood University
mitul.bhatia2024@nst.rishihood.edu.in

Phase I draft — revised November 8, 2026.
Individual contributions below are proposed and await completion.

***Abstract*—This project demonstrates a simplified Harvard memory organization in Verilog and compares it with shared single-port memory. Both designs use the same controller, program, initial data, clock period, and total storage. A three-instruction program loads a byte, stores it at another address, and halts. Harvard memory completes in four cycles and shared memory in six, with identical final data. Waveforms show instruction fetching overlapping data accesses only in the Harvard model. Six simulation scenarios verify memory separation, control signals, address mapping, reset, and correct results.**

***Index Terms*—Harvard architecture, shared memory, Verilog, instruction fetch.**

I. Introduction

Instructions tell a controller what operation to perform; data contains the values being used. Separate instruction and data paths allow the next instruction to be fetched while the current instruction accesses data. A shared single-port memory must serve those accesses in different cycles. Our goal is to demonstrate this difference with a small controller and an easily checked program. The project covers instruction/data separation, address decoding, memory control, waveforms, and architecture comparison. The optional shared lower-level memory is omitted.

II. Literature Review

Microchip's enhanced mid-range PIC documentation describes independent instruction and data buses and a two-stage pipeline: the next instruction is fetched while the current instruction executes [1]. Its AVR core documentation also describes separate program/data memories and instruction prefetch during execution [2]. These examples motivate our separate paths and one-register fetch/execute overlap. Our model uses its own small instruction set and does not emulate either processor. Icarus Verilog provides the compilation and simulation flow used to check the RTL [3].

III. Proposed Solution

A. Memories and Controller

Harvard mode uses a 16×8-bit instruction memory and a 16×8-bit data memory. Shared mode uses one 32×8-bit memory with a single address port. Both have 32 bytes in total. The shared address decoder maps instruction offsets to physical addresses 0–15 and data offsets to addresses 16–31. Fig. 1 shows both organizations.

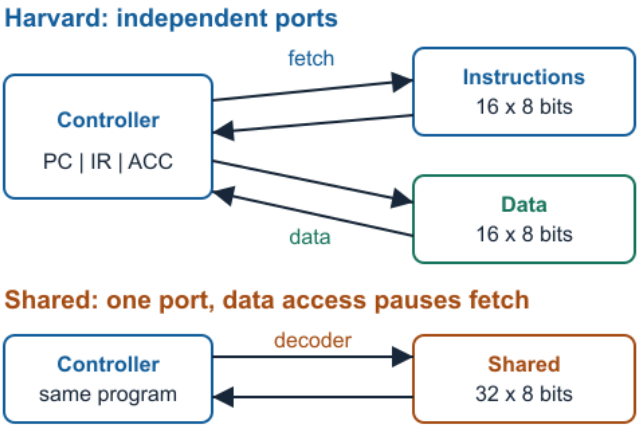


Fig. 1. Same controller and capacity, with independent or shared ports.

The controller contains a four-bit program counter (PC), an eight-bit instruction register with a valid bit, and an eight-bit accumulator (ACC). PC advances only when a fetch is accepted. The instruction register holds the current instruction while PC addresses the next one.

B. Instructions and Timing

The instruction's upper nibble selects the operation and its lower nibble selects a data address. LOAD (`1a`) reads data[a] into ACC; STORE (`2a`) writes ACC to data[a]; HALT (`Fa`) stops. NOP (`0a`) performs no data operation. Other opcodes stop with a fault flag. The low nibble is ignored by NOP and HALT.

Reads are combinational; LOAD captures data and STORE commits its write at the rising clock edge. A synchronous active-high reset clears the controller, suppresses memory requests, and retains memory contents. The simulation uses a 10-ns clock period. This is a test setting, not a measured hardware frequency. Programs must fit in 16 instruction bytes and end in HALT; the four-bit PC otherwise wraps.

In Harvard mode, fetching the next instruction can overlap the current LOAD or STORE. In shared mode, data has priority: fetch is blocked, PC holds, and the instruction-register

TABLE I
MEASURED COMPLETION CYCLES FOR PASSING SCENARIOS.

Scenario	Harvard	Shared
LOAD 0; STORE 1; HALT	4	6
Address boundaries and dependencies	6	10
Two NOPs and HALT	4	4
HALT only	2	2
Invalid opcode	2	2
Restart of longer copy sequence	8	14

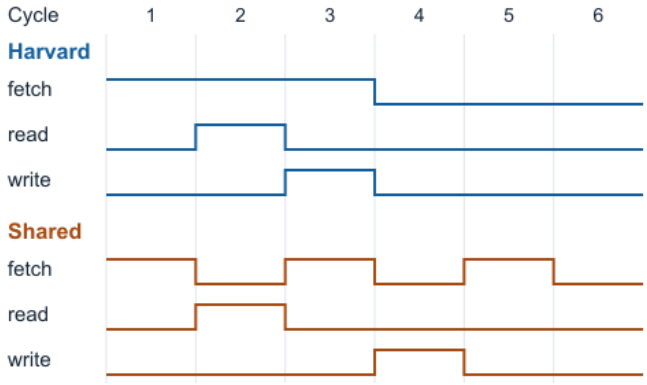


Fig. 2. Demo controls sampled before each rising edge. In cycles 2 and 3, Harvard fetch overlaps data read and write respectively.

valid bit clears. The next cycle fetches the delayed instruction without repeating the previous operation. HALT suppresses all further memory traffic.

IV. Prototype and Verification

The main program is `LOAD 0; STORE 1; HALT`, encoded as `10 21 F0`. Initially data[0] is 7 and all other data is zero. Both designs must finish with ACC=7, data[0]=7, and data[1]=7, with all other data and all instructions unchanged.

Running `make demo` displays each cycle and creates a short VCD waveform. Running `make test` checks six scenarios: the main demo, boundary addresses, no data accesses, HALT only, an invalid opcode, and reset/restart during a longer copy sequence. All six pass using Icarus Verilog 13.0.

The testbench checks every final data word, preservation of instructions, exact read/write counts, PC advance/hold behavior, exclusive shared-port ownership, and inactivity after HALT. Boundary tests include a STORE immediately followed by a LOAD of the same address. Reset tests check that completed writes are retained. The supplied hex images are verified before testbench memory changes.

V. Preliminary Results and Discussion

Table I shows measured simulation counts. Counting starts at the first rising edge after reset release and includes HALT execution. Reset cycles are excluded.

In cycle 1, both designs fetch LOAD. In cycle 2, Harvard executes LOAD and fetches STORE; in cycle 3, it executes STORE and fetches HALT. HALT executes in cycle 4. Shared memory blocks fetch during LOAD in cycle 2 and STORE in cycle 4, completing HALT in cycle 6. Fig. 2 shows these two overlap opportunities and two blocked shared fetches. Both designs produce the expected value 7.

The comparison isolates single-port contention under the stated memory model. The no-data test has equal cycle counts, demonstrating that this benefit depends on the workload. These results do not establish a universal processor speedup. Synthesis, FPGA resources, and hardware timing have not been measured.

VI. Individual Contributions

The following ownership is proposed. Before submission, each member must replace their entry with completed work and supporting commits from their own GitHub account.

- **Akshat Chauhan:** instruction memory, PC, and fetch checks.
- **Ajeesh Amreet:** data memory, read/write control, and data timing.
- **Vaibhav:** shared memory, address decoding, and fetch stalls.
- **Hardik Maheshwari:** testbench, waveforms, and verification.
- **Mitul Bhatia:** integration, results comparison, report, repository organization, and video assembly.

VII. Conclusion

The prototype meets the technical scope with separate Harvard paths, correct memory controls, simulation waveforms, and a shared-memory comparison. The short demo takes four versus six cycles with equal results. Remaining Phase I work is team review, completed individual contributions, GitHub publication, and recording the video. Phase II will extend the report and maintain repository history for the interview.

References

[1] Microchip Technology, "8-bit PIC MCU enhanced mid-range architectural overview," *Developer Help*, Jan. 23, 2024. [Online]. Available: https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8bit-pic/enhanced-family/mrarch/. Accessed: Nov. 8, 2026.

[2] Microchip Technology, "AVR CPU core," *Developer Help*, Nov. 10, 2023. [Online]. Available: https://developerhelp.microchip.com/xwiki/bin/view/products/mcu-mpu/8-bit-avr/structure/core/. Accessed: Nov. 8, 2026.

[3] S. Williams and contributors, "Getting started with Icarus Verilog," *Icarus Verilog documentation*. [Online]. Available: https://steveicarus.github.io/iverilog/usage/getting_started.html. Accessed: Nov. 7, 2026.